ZAALIMA DEVELOPMENT · DATA ANALYTICS INTERNSHIP

Week 2 • Day 2

Data Preprocessing for Machine Learning

Applied One-Hot Encoding, StandardScaler, and created a stratified 80/20 train/test split

DATE	INTERN NAME	PROJECT	MENTOR
Day 9 of 28 | Week 2	______________________ ______	Customer Churn Prediction & LTV Engine	______________________ ______

■ Technologies Used Today

Python	Pandas	Scikit-Learn	NumPy	joblib

■ Today's Key Metrics

5,634	1,409	26.5%	~47
Training Rows	Test Rows	Churn % Both Sets	Features After OHE

■ Day Objectives

- ■ Apply One-Hot Encoding to multi-class categorical columns
- ■ Scale numeric features using StandardScaler
- ■ Create stratified 80/20 train/test split preserving churn ratio
- ■ Save scaler and feature column list for API use later
- ■ Understand and prevent data leakage

■ Tasks Completed Today

1. Created src/features/preprocessing.py with preprocess_for_ml() function
2. Dropped non-ML columns: customerid, loyalty_segment

3. Applied pd.get_dummies() to 4 multi-class columns: internetservice, contract, paymentmethod, multiplelines

4. Confirmed shape after OHE: (7043, ~47 columns)

5. Separated target: y = df['churn'], X = all other columns

6. Created stratified 80/20 split: X_train (5634), X_test (1409) with matching churn %

7. Verified: y_train churn = 26.5%, y_test churn = 26.5% — stratification confirmed

8. Fitted StandardScaler on X_train ONLY (prevent data leakage)

9. Transformed both X_train and X_test using the fitted scaler

10. Saved models/scaler.pkl using joblib.dump()

11. Saved models/feature_columns.pkl — list of column names in correct order

12. Committed all files to GitHub

■ Concepts Learned

- One-Hot Encoding: why 'Month-to-month' must become a separate 0/1 column
- Label Encoding vs OHE: tree models can use label, linear models need OHE
- Data leakage: fitting the scaler on test data would give artificially inflated metrics
- Stratified split: ensures both train and test have the same 26.5% churn ratio
- random_state=42: makes the split reproducible — same result every run
- joblib.dump/load: fast way to save and load Python objects (models, scalers)
- Why feature column order must be saved — models expect features in exact same order

■ Tools / Software Used

Tool / Library	Version / Notes	Status
Scikit-Learn	1.3.2	✓ train_test_split, StandardScaler
joblib	1.3.2	✓ Model serialisation

■ Outputs / Deliverables Produced

❖ src/features/preprocessing.py — full preprocessing pipeline
❖ models/scaler.pkl — fitted StandardScaler (from training data only)
❖ models/feature_columns.pkl — list of 47 feature column names in correct order
❖ Train set: 5,634 rows, Test set: 1,409 rows
❖ Both sets have 26.5% churn (stratification verified)
❖ GitHub commit with preprocessing module and model files

■ Problems Encountered & Solutions

Problem Faced	How I Solved It

Shape mismatch error after OHE between train and test	Applied get_dummies() to the full dataset BEFORE splitting, ensuring identical columns in both train and test sets

■ GitHub Commit Record

Commit Message	Week2-Day2: Preprocessing — OHE, StandardScaler, stratified split, scaler and feature cols saved
Branch	main
Files Changed	src/features/preprocessing.py, models/scaler.pkl, models/feature_columns.pkl
Commit Hash	(fill after push)

■ End-of-Day Submission Checklist

■ OHE applied to 4 columns	■ Train: 5,634 rows
■ Test: 1,409 rows	■ Both sets: 26.5% churn
■ Scaler fitted on train only	■ models/scaler.pkl saved
■ feature_columns.pkl saved	■ preprocessing.py committed
■ No data leakage confirmed	■ Day 9 commit on GitHub

■ Self Assessment

8/10	6	Week 2, Day 2	9/28
Self Rating	Hours Spent	Progress	Day in Program

■ Key Learnings & Reflections

★ Never fit the scaler on test data — this is data leakage and makes metrics unreliable
★ Stratified split is essential for imbalanced datasets — random split could skew churn %
★ Always save feature_columns.pkl alongside the model — the API needs to know the exact feature order

★ pd.get_dummies on the full dataset before splitting is safer than separate OHE

■ Plan for Tomorrow

→ Train Logistic Regression as the baseline model
→ Calculate accuracy, precision, recall, F1-score, and AUC-ROC
→ Plot confusion matrix and ROC curve
→ Save models/logistic_regression.pkl

INTERN SIGNATURE	MENTOR SIGNATURE	DATE SUBMITTED
______________________________	______________________________	______________________________